# GSA

## PSSMs: Position specific scoring matrices (Searching)

identifying that a sequence belongs to the family, often allows inference about its function.
Capturing signals common to the members of a sequence family is best done by a statistical approach based on multiple alignment of the sequences in the family

eg: Family, GTAT, GGCG, TTCG, GTCA, GTTA

| | | | |
|---|---|---|---|
| G | T | A | T |
| G | G | C | G |
| T | T | C | G |
| G | T | C | A |
| G | T | T | A |

⇒ PSSM:

| | | | | |
|---|---|---|---|---|
| A | 0 | 0 | 0,2 | 0,4 |
| C | 0 | 0 | 0,6 | 0 |
| G | 0,8 | 0,2 | 0 | 0,4 |
| T | 0,2 | 0,8 | 0,2 | 0,2 |

↳ scored by sum up the scores of the cases of the positions

## Position Specific Scoring Matrix:

eg: high PSSM-score in some region of a seq often identicales a possible biological relationship of this seq to the family!

Representation of a multiple Alignment of related sequences.

- $M: A \times \{1, \dots, m\} \to \mathbb{R}$, A alphabet, m is the length of M.

|A| x m - Matrix, row: frequency of that nucleotide at positions 1-m

- M a PSSM of length m: $score(w, M) = \sum_{k=1}^{m} M(w[k], k)$

- PSSM Searching Problem: find all positions $j$, $1 \leq j \leq n-m+1$ in S with $|S| = n$ s.t. $score(S[j \dots j+m-1], M) \geq t_h$

# GSA

## PSSMs

- Simple algorithm to solve PSSM search problem:
  - slide along sequence and computes $score(w, M)$
    $\forall w = S[j \dots j+m-1], j, 1 \le j \le n-m+1$ $\Rightarrow O(mn)$ time

- Look ahead scoring: ↑ improves ↑
  - Stop calculation of $score(w, M)$ when threshold can not be reached anymore.

$$pfx\,score_d(w, M) = \sum_{k=1}^{d} M(w[k], k)$$

$$max_d = max\{ M(a, d) \mid a \in A \}$$

$$\sigma_d = \sum_{k=d+1}^{m} max_k \qquad \text{// could max}^{d} \text{ entries} \longrightarrow$$

$$th_d = th - \sigma_d \qquad \text{// intermediate threshold}$$

$\Rightarrow$

$$pfx\,score_d(w, M) \ge th_d, \quad \forall 1 \le d \le m$$

$\Leftrightarrow$ $score(w, M) \ge th$

↳ While scanning from left-to-right check if $pfx\,score_d(w, M) \ge th_d$
  ↳ if not stop!
  ↳ runs in $O(kn)$ time, $k$ is # of PSSM-positions per sequence that actually evaluated
  ↳ Worst case $O(mn)$

# GSA

## Länge längste gemeinsame Subsequenz

$$lcs_{len}(u,v) = \frac{m+n - edist_\delta(u,v)}{2}$$

## Edit + Vertauschen

$ab \to ba$ , $a,b \in A$

- $\delta'(ab \to ba) = 0$ , $a,b \in A$
- $\delta'(ab \to cd) > 0$ , $a,b,c,d \in A$ , $a \neq d$ oder $b \neq c$

add edges:

$$(i-2, j-2) \xrightarrow{u[i-1]u[i] \to v[i-1]v[i]} (i,j)$$

Wie zwei $\delta$ replacements!

cost: $\delta'(u[i-1]u[i] \to v[i-1]v[i])$

$$\delta'(ab \to cd) = \begin{cases} 0, & a = d \text{ & } b = c \\ \infty, & \text{else} \end{cases}$$

$\otimes_0^2$

↳ dann

$$E_\delta(i-2, j-2) \leftarrow \delta'(u[i-1]u[i] \to v[i-1]v[i]) \quad \forall i,j \overset{\geq 2}{>} 1$$

# Ch 5 ☆

**Problem eg 8**

$u = agtgcacacatc$
$v = atcacacttagc$

$\Psi_{lr}(u,v) = \{ag, t, gc, a, cacca, t, c\}$
$\Psi_{rl}(u,v) = \{a, g, t, g, caca, c, atc\}$

$\Psi_{lr}(v,u) = \{atc, a, cac, t, t, agg, c\}$
$\Psi_{rl}(v,u) = \{a, t, cacac, t, ga, gc\}$

**q-gram code**

$$\overleftarrow{w} = \sum_{i=1}^{q} \overleftarrow{wc(w[q-i+1])} \cdot r^{q-i}$$

$wc(b)$: $A \to T, T \to A, C \to G, G \to C$

$$\overleftarrow{xc} = (\overleftarrow{ax} - \overleftarrow{wc(a)} r^{0}) \cdot r^{-1} + \overleftarrow{wc(c)} \cdot r^{q-1}$$

$$\overleftarrow{wc(b)} = 3 - \overleftarrow{b}$$

# GrSA

## Speichereffiziente Alignments

Multi-Eops Liste: $[x_1 y_1 \dots x_k y_k]$

$x_i \in \{R, D, I\}$, $y_i > 0$

zwei gleich lange Listen:

$[x_1 \dots x_k]$

$[y_1 \dots y_k]$

, u & v & Liste $\Rightarrow$ Alignment

## qdist & edist

eg: 1) $q\text{gdist}(u,v) > \text{edist}_S(u,v)$

u = RYRRYRYRR

v = RRRYRYRYRR

$q = 3$ $\quad 3 > 2$

2) $q\text{gdist}(u,v) < \text{edist}_S(u,v)$

u

v $\quad q = 2 \quad 1 < 2$

3) $q\text{dist}(u,v) = \text{edist}_S(u,v)$

u = ACCC $\quad 0 = 0$

v = ACCC

$q = 3$

# GASA

## Significance of local alignments

**Question:** Is this local alignment occuring by chance or not.

→ Quantifying the statistical significance of a local alignment, based on a simple model of random sequences.

→ E-Value: statistical significance.

↳ The smaller the E-value, the more significant the local alignment

→ Determine E-Value for a local alignment in which the number of mismatches is smaller than a threshold

→ Use hamming distance of the aligned pair of substrings

↳ $H(x,y) = |\{i \mid 1 \leq i \leq |x|, x[i] \neq y[i]\}|$ // $|x| = |y|$

**Problem:** $u, |u| = m$, $v, |v| = n$, Determine pair of substrings in $u$ and $v$ allowing for a maximum number $k \geq 0$ of mismatches in these substrings. $k$ is mismatch threshold

# GSA

## Significance of local Alignment

## Local k-mismatch Alignment

Local k-mismatch Alignment of $u$ and $v$ (k-LMA) of length $\ell$ is a triple $(\ell, i, j)$ s.t.:

- $k < \ell$ i.e. #mismatches < length
- $1 \le i \le m - \ell + 1$, $1 \le j \le n - \ell + 1$:
  $u[i \dots i+\ell-1]$ is substring of $u$
  $v[j \dots j+\ell-1]$ is substring of $v$
  ($\ell$)
- $H(u[i \dots i+\ell-1], v[j \dots j+\ell-1]) \le k$ $\hat{=}$ At most k mismatches

Now: Extend each alignment to both sides maximally!

- left-maximal: if $i=1$ or $j=1$ or $u[i-1] \neq v[j-1]$
- right-maximal: if $i+\ell=m$ or $j+\ell=n$ or $u[i+\ell] \neq v[j+\ell]$
- $(\ell, i, j)$ is maximal if both

$\Rightarrow$ E-value = $| \{(\ell', i', j') \mid (\ell', i', j')$ is $k'$-LMA in $u'$ and $v'$, $\ell \le \ell', k' \le k\} |$

$u'$ & $v'$ are random sequences of $|u'| = |u|, |v'| = |v|$

↳ # of local alignments of the same length or longer and with the same or a fewer number of mismatches, that occur one would expect to find between two random sequences of the same size as $u$ & $v$

Model of random sequences, uniform Bernoulli Model.

$r = |A|$, $p(\text{char}) = \frac{1}{r}$

# GS*

## Restricting to exact matches
~~Local & Gumbel Regime~~

When $k = 0$ $\Rightarrow$ maximal exact matches.

- $\forall$ pair of seq's $u$ & $v$, the # of maximal exact matches of length $\geq \ell$ equals the number of left-maximal exact matches of length exactly $\ell$

- $E[\#\text{ events}]$ : expected number of events
- Events are maximal exact matches of length $\geq \ell$ and left-maximal matches of length $= \ell$.
- $E[\#\text{ events}]$:
  - → each possible position where an event occurs
  - → determine probability that the event occurs at that position
    - ↳ $Pr[\text{event occurs at position } p]$

- $E[\#\text{events}] = \sum_{\text{position } p} Pr[\text{event occurs at position } p]$

$\Rightarrow E[\#\text{ of maximal exact matches of length} \geq \ell] = mnp^{\ell}(1-p)$

# GASA

## Allowing mismatches

$k > 0$;

compute the # of choices of $k$ positions with mismatches from $\ell$ possible positions in the seq's.

$X$ of size $\ell$

$$\binom{\ell}{k} = \frac{\ell!}{k!(\ell-k)!} \qquad \text{subsets } Y \subseteq X, |Y| = k$$

- $\Pr[\,H(x,y) = k\,] = \binom{\ell}{k} p^{\ell-k} (1-p)^{k}$
- $E[\,\#\text{ of maximal} \le k\text{-MMs of length} \ge \ell\,]$
  $= m n \binom{\ell}{k} p^{\ell-k} (1-p)^{k+2}$

# GSA

## Linear Space Alignment Algorithm

→ reduce space requirement

~~$\mathcal{O}(mn)$ Space~~ ⇒ $\mathcal{O}(m+n)$ Space
$\mathcal{O}(\min\{m,n\})$

to compute a single optimal alignment

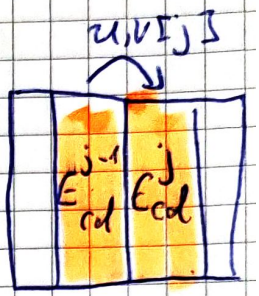
recap:

$$E_S(i,j) = edist_S(u[1..i], v[1..j])$$

$0 \leq j \leq n$:

$$E^j_{col} = E_S(0,j), \ldots, E_S(m,j)$$

↪ Distance Columns.




⇒ Compute $E^0_{col}, \ldots, E^n_{col}$ s.t. only two columns are saved at any time.

Alg 0:

```
first ED tab column (E, u):
        E(0) ← 0
        for i = 1 upto |u| do E(i) ← E(i-1) + δ(u[i] → ε)
end
next ED tab column (E, E', u, b):
        E'(0) ← E(0) + δ(ε ← b)
        for i = 1 upto |u| do E'(i) ← min { E'(i-1) + δ(u[i] → ε)
                                          { E(i) + δ(ε → b)
                                          { E(i-1) + δ(u[i] → b)
end
```

```
evaluate ED tab column (E, u, v):
        first ED column (E, u) & allocate E' : |u| + 1
        for j ← 1 upto |v| do next ED col. (E, E', u, v[j]); E ← E'
end
```

# GSA

## Linear Space Alignment Algorithm

distance only algorithm calls evaluate ED columns ($E, u, v$)

↳ $E_S(m,n) = E^u_{col}(m)$ in $O(m)$ space / $O(\min\{m,n\})$ space

⇒ Can be improved in using one vector and two scalars!

## Applying Divide and Conquer

optimal Alignment ⇔ path in edit-Graph $G_S(u,v)$ $(0,0) \to (m,n)$

↳ can not traceback

**Idea:** linear space ~~algorithm~~ optimal alignment algorithm:
keep some of the history and apply a divide and conquer strategy
to obtain a representation of an optimal alignment from this history

**Alg:**

- Consider column $j_0 = \lfloor n/2 \rfloor$
  - ↳ find node $(i_0, j_0)$, that is on the min path $(0,0) \to (m,n)$ in $G_S(u,v)$
  - ↳ Call recursively for $(0,0) \to (i_0,j_0)$ & $(i_0,j_0) \to (m,n)$
- Each Recursion Step delivers a cornpoint.
- up to Recursion level $k$, $k \leq \log_2 n$
- $cps(k) = 2^k - 1$ cornpoints, eg: $cps(\log_2(n)) = n-1$

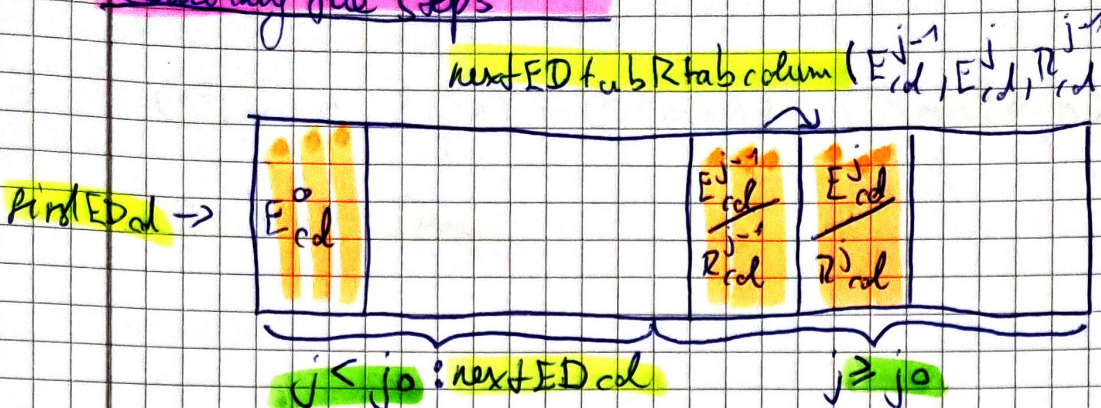
# GSA

## Divide and Conquer

- Node is in middel of $(0,0) \to (m,n)$, iff $(-, j_0)$, $j_0 = \lfloor n/2 \rfloor$
- find $i_0$, $0 \le i_0 \le m$ s.t., $(i_0, j_0)$ is on min path from $(0,0) \to (m,n)$.
  - compute $(m+1) \times (n+1)$-table $R_{j_0}$ storing row indices.
    - ↳ Columns are referred to row index columns.
  - $\forall i, 0 \le i \le m$, $\forall j, 0 \le j \le j_0 - 1$ : $R_{j_0}(i,j)$ is undefined.
  - $\forall j, j_0 \le j \le n$, $\forall i, 0 \le i \le m$ : $R_{j_0}(i,j) = i_0$
    - → then there is a min path from $(i_0, j_0) \to (i,j)$.
    - $R_{j_0}(m,n) = i_0 \to (i_0, j_0)$ is a cross point!
  - $R_{j_0}(i,j) = \begin{cases} \text{undefined} & , j < 0 \le j \le j_0 - 1 \\ i & , \text{else if } j \ne j_0 \\ R_{j_0}(i, j-1) & , \text{else if } E^j_{col}(i) = E^{j-1}_{col}(i) + \delta(\varepsilon \to v[j]) \\ R_{j_0}(i-1, j-1) & , \text{else if } E^j_{col}(i) = E^{j-1}_{col}(i-1) + \delta(u[i] \to v[j]) \\ R_{j_0}(i-1, j) & , \text{else if } E^j_{col}(i) = E^{j-1}_{col}(i-1) + \delta(u[i] \to \varepsilon) \end{cases}$
  - One computes $R^j_{j_0}$ along with $E^j_{col}$ → $O(\min\{m,n\})$ space

# Combining the steps

## Combining the steps




⇒ Compute $E_S$ & $R_{j_0}$, given middle column $j_0$

- $E^{n}_{col}$ & $R^{n}_{j_0}$, $j_0 = \lfloor \frac{n}{2} \rfloor$, $edist_S(u,v) = E^{n}_{col}(m)$ & $(i_0, j_0) \neq i_0 = R^{n}_{j_0}(m)$
- Store $i_0$ at index $j_0$ of a table $C$, $|C| = n$
  - ↳ ~~for~~ Crosspoint Table
  - ↳ encodes an optimal alignment
  - ↳ $C(0) = 0$ & $C(n) = m$

**Alg:**

evaluate crosspoints $(C, i, j, p, q)$

if $q \geq 2$ :

$j_0 \leftarrow \lfloor q/2 \rfloor$

evaluateallcolums $(E, R, j_0, u[i \dots i+p-1], V[j \dots j+q-1])$ — global vectors

$i_0^* \leftarrow R[q]$

$C(j-1+j_0) \leftarrow i-1+i_0$

evaluate crosspath $(C, i, j, i_0, j_0)$

evaluate crosspoints $(C, i+i_0, j+j_0, p-i_0, q-j_0)$

end

evaluatecrosspoints $(C, 1, 1, m, n)$

$O(m)$

# GSA

## Space Requirement

$O(m+n+\log n) = O(m+n) = O(\min\{m,n\})$ Space

$O(mn)$ Time

# GSA

## Multiple Sequence Alignment

Compare $k \geq 2$ sequences.

usually for a set of sequences that are assumed to be homologous ~~features~~.

Goal: detect homologous residues

Score MSAs.

- set $S$ of $k$ sequences $s_1, \dots, s_k$ & $n_i = |s_i|$
  - $\mathcal{A}$ is a gap symbol $\mathcal{A}' = \mathcal{A} \cup \{-\}$
- delgap : $(\mathcal{A}')^* \to \mathcal{A}^*$ :

  $delgap(a) = a$, $delgap(-) = \varepsilon$

  $delgap(w[1..n]) = delgap(w[1]) \dots delgap(w[n])$

## MSA

Multiple Sequence Alignment of $S$ is a sequence of $k$ strings $(s_1', \dots, s_k')$ s.t.:

1) $\ell \geq 1$ s.t. $s_i' \in (\mathcal{A}')^\ell$, $\forall i, 1 \leq i \leq k$
2) $delgap(s_i') = s_i$
3) $j$, $1 \leq j \leq \ell$, there is at least one $i$, $1 \leq i \leq k$, $s_i'[j] \neq -$

$$\max\{n_i \mid 1 \leq i \leq k\} \leq \ell \leq \sum_{i=1}^{k} n$$

# MSA

## Scoring by consensus costs

The consensus of a column is a non-gap symbol that occurs most often in the column.

- $count(j, a) = |\{i \mid 1 \le i \le k, s_i'[j] = a\}|$ , $a \in \mathcal{A}$
  - ↳ # of occurences of $a$ in column $j$
- cons is a seq is a consensus of $(s_1', \dots s_k')$. $\forall j \le j \le l$.
  $count(j, cons[j]) \ge count(j, a)$ $\forall a \in \mathcal{A}$
- $dist(cons, (s_1', \dots, s_k')) = \sum_{j=1}^{l} (k - count(j, cons[j]))$

**Lemma**

$cons_1$ & $cons_2$ be two diff consensi of the same MSA, then
$dist(cons_1, (s_1', \dots, s_k')) = dist(cons_2, (s_1', \dots, s_k'))$

$\Rightarrow$ $conscost(s_1', \dots, s_k') = dist(cons, (s_1', \dots, s_k'))$
↳ the smalle the better!

MSA-problem: with consensus costs consists of finding the MSA $(s_1', \dots, s_k')$ of the given seq's $s_1, \dots, s_k$ s.t.
$\min conscost(s_1', \dots, s_k')$

# GSA

## Scoring by sum of pairs

pairwise distance:

$\delta(\alpha \to \beta) \geq 0$ & $\delta(- \to -) = 0$

Calculate the sum of scores of all pairs of the chars and gaps in each column of the MSA.

SP-column score: $1 \leq j \leq l$

$$s_{sp}(s_1'[j], \ldots, s_k'[j]) = \sum_{i=1}^{k-1} \sum_{r=i+1}^{k} \delta(s_i'[j] \to s_r'[j])$$

SP-alignment:

$$s_{sp}(s_1', \ldots, s_k') = \sum_{j=1}^{l} s_{sp}(s_i'[j], \ldots, s_k[j])$$

MSA-Problem with SP-scoring:

Find an MSA of the given sequences with min SP-alignment score

$\Rightarrow s_{opt\text{-}sp}(s_1, \ldots, s_k)$

# GSA

## Compute optimal MSA

use edit recurrence:

- compute $(n_1+1) \times \dots \times (n_k+1)$ - table $M$ s.t.:
  $(q_1, \dots, q_k) \in [0, n_1] \times \dots \times [0, n_k]$
- $M(q_1, \dots, q_k)$ is the score of the optimal MSA for ~~each~~
  - the prefixes $S_1[1..q_1], \dots, S_k[1..q_k]$.
- each column: $(d_1, \dots, d_k) \in \{0,1\}^k \times \dots \times \{0,1\}^k \setminus \{(0, \dots, 0)\}$
  $d_i = 0$ : $i$th char is a gap symbol
  $d_i = 1$ : $i$th char is $S_i[q_i]$
- $M(q_1, \dots, q_k) =$ prev
  $\min\{ M(q_1 - d_1, \dots, q_k - d_k) +$ next
  $\text{colcost}(\varphi(q_1, d_1), \dots, \varphi(q_k, d_k)) \mid (d_1, \dots, d_k) \in \{0,1\} \times \dots \times \{0,1\},$
  $(d_1, \dots, d_k) \neq (0, \dots, 0),$
  $q_1 \geq d_1, \dots, q_k \geq d_k \}$

$$\varphi(q_i, d_i) = \begin{cases} S_i[q_i], & d_i = 1 \\ -, & \text{else} \end{cases}$$

Define Recursion over

$k=2$ $q_1=0, q_2 \geq 1$ ; $q_1 \geq 1, q_2 = 0$ ; $q_1 \geq 1, q_2 \geq 1$

min over all $\begin{cases} -1 \;\; 0 \;\; S\dots \;\; - \\ 0 \;\; -1 \;\; - \;\; S\dots \\ -1 \;\; -1 \;\; S\dots \;\; S\dots \end{cases}$

↓ binär hoch zählen!

# GSA

## Capale MSA

- It has $z = \prod_{i=1}^{k} (n_i + 1)$ entries
- each entry at least $2^k - 1$ computations
- consensus columns $O(k)$
- SP-scoring column cost $O(k^2)$
- organized in one dimensional array of size $z$.
  entry: distance / bitvector of $k$ bits.
  ↳ which gave raise to min costs
  ↳ capule address of an entry in $O(k)$ time
- → each entry is capuled in $O(f(k) \cdot 2^k)$ time

$$f(k) = \begin{cases} k & \text{, consensus} \\ k^2 & \text{, SP-scoring} \end{cases}$$

⇒ Total $O(z \cdot f(k) \cdot 2^k)$ time $O(z)$ space

- traceback $O(kl)$ time

# MSA

## Combining pairwise alignments

since exact is very costly

approximative solution.

**Idea:** construct MSA from pairwise alignments of the seq's

## Compatible

$S = \{s_1, \dots, s_k\}$ a set of sequences, and $S'$ be a subset of $S$.

Consider an MSA $Al_S = (s'_1, \dots, s'_k)$ of $S$.

Projection of $Al_S$ with respect to $S'$ is the MSA $proj(Al_S, S')$:

1) delete all rows in $Al_S$ which do not correspond to a seq in $S'$
2) delete all columns with only gaps

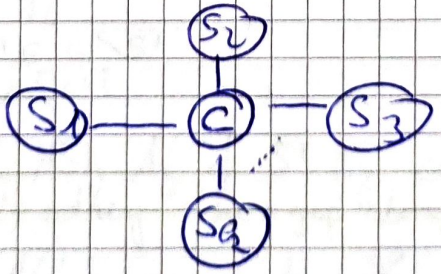
$Al_S$ is compatible with $Al_{S'}$ of $S'$ if $Al_{S'} = proj(Al_S, S')$

## Alignment Tree

S a set of seq's.

Alignment tree for S is a labeled undirected tree, where the node set is S and each edge $(s,t)$ in the tree is labeled by an optimal pairwise alignment of the seq's s & t.

Star alignment: center node c



# MSA

## Scoring pairwise alignments

Star alignment method:

1) identify the center seq.
2) construct from the optimal pairwise alignments a MSA that is compatible with the pairwise alignments

→ Once a gap always a gap!

$c' = ATG$ $\quad$ $c'' = A\ominus TG$
$s_1' = A-G$ $\quad$ $s_2'' = AA--$

↳ $c''' = A\ominus\ominus TG$
$s_1''' = A--G$ → resulting MSA is not optimal
$s_2''' = AA--$

## Alg.

compute c:

for $\forall s \in S$ do:
  for $\forall t \in S \setminus \{s\}$ do:
    compute an optimal alignment of s and t with dis $\delta_{opt}(s,t)$
  end
  $totalscore(t) \leftarrow \sum_{s \in S \setminus \{t\}} \delta_{opt}(s,t)$
let $c \in S$ s.t. $totalscore(c) \leq totalscore(s)$ $\forall s \in S$
let T be a star alignment with c as center

compatible MSA:

choose $s \in S$
let AL be an optimal alignment of c and s
$S' \leftarrow \{c, s\}$
while $S' \neq S$ do
  choose $s' \in S \setminus S'$
  $S' \leftarrow S' \cup \{s'\}$

  Determine an MSA AL' of S' by combining AL with an optimal alignment of c and s', adhering "once a gap always a gap"
    ↳ AL' is compatible with AL

  $AL \leftarrow AL'$
end

# MSA

## Scoring pairwise alignments

MSA is well approximated the ~~SP~~ optimal optimal SP-alignment if:

$\delta : A' \times A' \to \mathbb{R}_{\geq 0}$ is good if:

1) $\forall\ a \in A',\ \delta(a,a) = 0$

2) $\forall\ a,b,c \in A',\ \delta(a,c) \leq \delta(a,b) + \delta(b,c)$

## Lemma

$\delta$ a good cost function, $S = \{c, s_1, \dots, s_k\}$.

$T$ a star alignment tree with center $c$,

MSA of $S$ of length $l$ compatible with $T$ $(c', s_1', \dots, s_k')$

$\forall i,j,\ 1 \leq i,j \leq k$:

$$\delta(s_i', s_j') \leq \delta(s_i', c') + \delta(c', s_j') = \delta_{opt}(s_i, c) + \delta_{opt}(c, s_j)$$

## Theorem

$\delta$ a good cost function and $\delta_{SP}$.

given $S$ set $\{s_1, \dots, s_k\}$ and MSA $AL = (s_1', \dots, s_k')$ of $S$

$$\delta_{SP}(\underbrace{s_1', \dots, s_k'}_{\text{Star alignment}}) \leq (2 - \tfrac{2}{k}) \cdot \underbrace{\delta_{opt\text{-}SP}(s_1, \dots, s_k)}_{\text{optimal SP cost alignment}}$$

→ Star alignment a good approximation of the optimal SP alignment.

↳ $\delta_{SP}$ is an upper bound for the optimal SP-alignment score

↗ using pairwise ↳ reduce search space for SP-costing model

$M(q_1, \dots q_a) \gg d > \delta_{SP}$

↳ strong because $\delta_{opt\text{-}SP} < \delta_{SP}$

# GSA

## Iterative Multiple Alignment

rooted tree with leaves $s_1, \ldots, s_k$

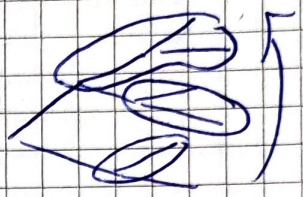
Iteratively combines alignments along the path of the tree.

I) $\forall$ pairs of sequences, compute optimal pairwise alignment and corresponding edit distance

II) ↳ compute a tree

III) combine pairwise alignments to larger alignments, guided by the branching order of the tree. → guide tree

$d(s_i, s_j)$ a distance between each pair



Advantage: (+) leads to fast algorithms and are applicable to many & long seq's

(+) Conservations in subfamilies that are aligned before other, less closely related sequences are added, lead to high quality MSAs

Disadvantages: (−) Errors in an early step cannot be eliminated later "once a gap always a gap"

(−) not well defined objective function which must be optimized during the iterative alignment method.

(−) guide tree remains represented in the alignment and might bias the result

# GSA

## Divide and Conquer Alignment

- Each sequence is cut in two after a suitable cut position somewhere close to its midpoint.
- -> the problem of aligning one family of sequences divided into the two problems of aligning two families of shorter sequences, the prefix and the suffix sequences.
- finally concatenate results.

# GSA

## Phylogenetic Analysis

Collection of species → determine & describe the evolutionary relationship between the species.

↳ branching process


## Phylogenetic Trees

- $X$ a finite set of taxa.
- A taxon $x \in X$ group of individuals defined in some way.

Phylogenetic Tree $T = (V, E, \lambda)$, connected graph $(V, E)$ without cycles, together with a one-to-one labeling $\lambda$ of the leaves by elements of $X$.

1) Rooted trees reflect the most basal ancestor of the tree in question
2) Unrooted trees do not imply a known ancestral root

→ binary if every internal node has degree 3, except $p$ with $p$ is root

## Edge length

describes putative order of speciation events that gave rise to the considered taxa

(+) positive length to each edge of the tree

## Methods

1) Distance Methods *
2) Maximum parsimony
3) Maximum likelihood approaches

→ $d: X \times X \times X \to \mathbb{R}_+^0$ , metric

# GSA

## Additivity and four point condition

### Additivity

$T_X$ a phylogenetic tree over $X$. $d: X \times X \to \mathbb{R}_+^0$.

$d$ is additive with respect to $T_X$ if it was obtained by adding up edge lengths on paths between the leaves of $T_X$

↳ $d: X \times X \to \mathbb{R}_+^0$ is additive, if there is a phylogenetic tree $T_X$ s.t. $d$ is additive with respect to $T_X$

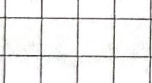
### Four Point Condition

$d$ is additive, iff $\quad w, x, y, z \in X$

$$d(w,x) + d(y,z) \leq \max\{d(w,y) + d(x,z), d(w,z) + d(x,y)\}$$

↳ decision is made in $\mathcal{O}(n^4)$

eg: euclidean distance is not additive



# GSA

## UPGMA

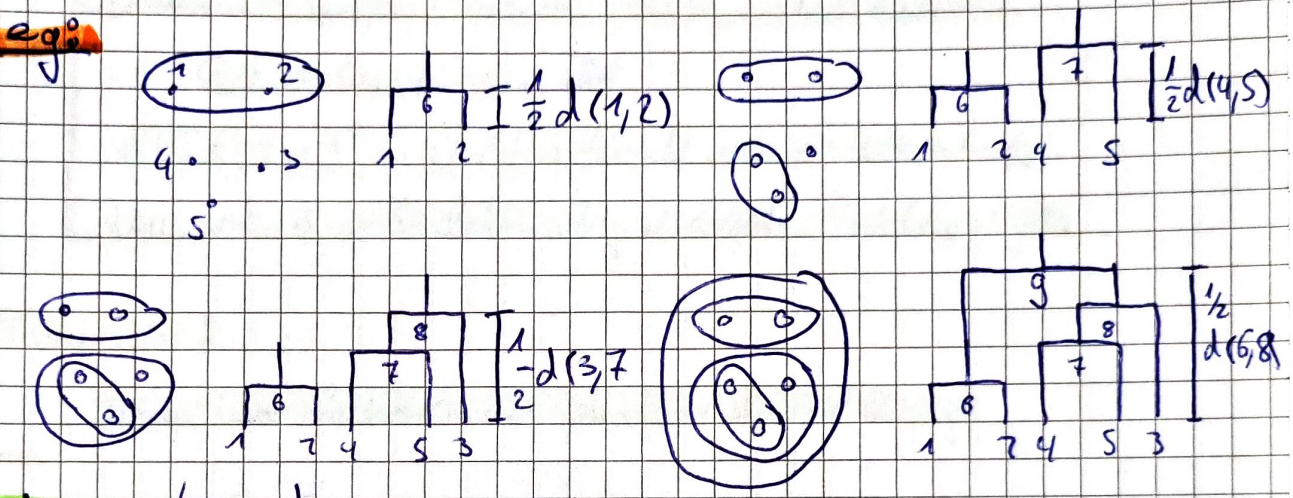
unweighted pair group method using arithmetic averages

→ rooted binary Tree $T_x$

→ clustering upwards: first clustering pair of leaves.

→ each node is given a height and the edge length are obtained as difference of heights of its two end nodes

eg:




### nearest pair:

1) $d(x,y)$, $\forall x,y \in X$

2) Cluster Distances $d(C_i, C_j)$, $C_i \subseteq X$ & $C_j \subseteq X$, $C_i \cap C_j = \emptyset$

$$d(C_i, C_j) = \frac{1}{|C_i|\,|C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x,y)$$

$$d(C_k, C_\ell) = \frac{d(C_i, C_\ell)\,|C_i| + d(C_j, C_\ell)\,|C_j|}{|C_i| + |C_j|}$$

$C_k = C_i \cup C_j$

# GSA

## UPGMA Algorithm

Initialization:

each taxon $x_i$ to its own $C_i$. Define one leaf of T for each taxon, placed at height zero

Iteration:

Determine $(C_i, C_j)$ which $d(C_i, C_j)$ is minimal.

↳ $C_k = C_i \cup C_j$.

$d(C_k, C_l)$ in const time for all existing clusters $C_l$.

New node k with childs i & j at height $\frac{1}{2} d(C_i, C_j)$.

Termination:

When two clusters remain place root at $\frac{1}{2} d(C_i, C_j)$

Runtime $O(n^2)$

→ biologically incorrect. ↯

~~→ we do not know~~

→ Simulation: known tree $T^0$ is used to generate artificial seq's & distances. Construct phylogenetic tree compare result T with $T^0$

# GSA

## Molecular clock hypothesis

UPGMA good all leaves have same distance to root p.

↳ is suitable for distance functions based on seq data that has evolved with a mutation rate constant over time.

evolutionary events at a constant rate = molecular clock hypothesis

& d is additive

## Ultrametric

↳ $d : X \times X \to \mathbb{R}_+^0$ ultrametric if

$\forall (x, y, z) \in X \times X \times X$, the dists $d(x,z), d(x,y)$ & $d(y,z)$ have one of:

1) all dist's are the same
2) two are equal & the remaining one is smaller

↳ decision in $\mathcal{O}(n^3)$

## Theorem

d is additive w.r.t. tree $T_X$

⇒ d is ultrametric if and only if every leaf in $T_X$ has the same distance from the root

↳ if both properties hold use UPGMA

# GSA

## Neighbor Joining

most widely used neighbor joining.

additive $d$ → unrooted tree $T$.

When the rate of evolution of the separate subtree under consideration varies.

$$N_{i,j} = d(i,j) - (r_i + r_j) \quad , r_i = \frac{1}{n-2} \sum_{x \in L} d(i,x)$$

$L$ is the current set of nodes in the tree, $n = |L|$.

- if $d$ is additive w.r.t $T$, then the two leaves $i$ and $j$ for $N_{ij}$ is minimal are neighbors in $T$

### Algorithm

Initialization:

Let $T$ consist of leaf nodes, one for each taxon. $L \leftarrow T$

Iteration:

Compute $N$. Pick a pair $i, j \in L$, s.t $N_{ij}$ is minimal.

Define new node $k$:

$$d(k,m) = \frac{1}{2}(d(i,m) + d(j,m) - d(i,j))$$

$\forall m \in L \setminus \{i,j\}$. Add $k$ to $T$ and construct edges.

$$d(i,k) = \frac{1}{2}(d(i,j) + r_i - r_j) \quad , r_i = \frac{1}{n-2} \sum_{x \in L} d(i,x)$$

$$d(j,k) = d(i,j) - d(i,k)$$

Remove $i$ & $j$ from $L$ and add $k$ to $L$.

Termination: $L$ only two nodes.

Add edges $i,j$ with $d(i,j)$

fast, useful and robust